

Conversión de tipos

Índice

Conversión de tipos primitivos o escalares	2
El caso de los caracteres	3
Promoción numérica	4
Conversión de tipos de referencia	5
Upcasting	5
Downcasting	5
Instanceof	6

Conversión de tipos primitivos o escalares

En un apunte anterior vimos los tipos de datos escalares (int, long, etc.). La conversión entre tipos primitivos es relativamente sencilla, siempre que se tengan en cuenta ciertos conceptos. Convertir desde un tipo “más chico” a uno “más grande” es la operación de conversión más sencilla:

```
int entero = 478;
long numero = entero;
```

Este es un caso trivial, ya que no debería haber problema en acomodar un número de 32 bits (int) en un espacio de 64 bits (long). Ocurre lo mismo con los números de punto flotante. Siempre que me mueva hacia un “contenedor más grande”, no habrá pérdida de información. A su vez, es posible pasar desde un tipo entero a uno de punto flotante. La parte decimal se llenará con ceros para completar el valor.

```
int entero = 241;
double numeroConComa = entero; // numero con coma será 241.0
```

El caso inverso sería pasar de un contenedor más grande a uno más pequeño. En estos casos puede haber pérdida de información de acuerdo al tamaño del valor de origen.

Por ejemplo:

```
long entero64 = 82233720368547758071;
int entero32 = (int)entero64; //entero32 vale ahora:
1486618623
```

Es por esto que hay que tener cuidado el momento de este tipo de conversiones, dado que puede causar efectos secundarios indeseados en nuestro sistema.

El caso de los caracteres

En el caso especial que se quiere convertir de un tipo byte a un tipo char, hay que tener en cuenta lo siguiente:

```
// el entero 165, al pasarlo a byte vale: -91 -> 0b10100101
byte caracterComoByte = (byte)165;

// esto da el caracter '\u0091' que es: 0b11111111 10100101 sin
signo
char caracterComoChar = (char) caracterComoByte;
// si voy a un "contenedor mas grande" entonces obtengo:
// 0b11111111 10100101 que es el numero entero: 65445
int caracterComoEntero = caracterComoChar;

//y al volver 65445 a byte, se pierden los bits más
significativos
//(porque me voy a un contenedor mas chico)
//entonces obtenemos: -91 -> 0b10100101
byte caracterComoByteNuevo = (byte)caracterComoEntero;
```

La representación binaria del número 165 es la misma que para el -91, excepto por la interpretación del bit que indica el signo (ver tipos de datos en el módulo anterior). El valor en tipo char resulta en un valor de carácter UNICODE. A su vez, al pasarlo a un tipo int quedaron los 8 bits menos significativos iguales, resultando en el mismo -91.

Es importante notar en este caso, que si todo esto hubiera arrancado desde un tipo char, los resultados no hubieran sido consistentes:

```
//aca 165 es se toma como entero, no como char
// al ponerlo en un tipo char, solo pierdo los bits de signo:
char caracterComoChar2 = 165;

//entonces obtengo: 0b 00000000 10100101, que es el mismo 165
// si me paso paso a un "contenedor mas grande"
```

```
// no pierdo ni gano información, solo se rellenan con ceros
// los bits mas singificativos => 0b00000000 10100101
int caracterComoEntero2 = caracterComoChar2; // este entero
es 165

//pero al convertirlo a byte SI pierdo información
//porque bits es un contenedor "mas chico" que int
byte caracterComoByte2 = (byte)caracterComoEntero; //-91
```

Entonces, es importante entender que para convertir de tipo byte a un char, es importante pasar primero por un int y luego convertir a char para no perder información en el camino.

Promoción numérica

Hay una serie de reglas que se aplican al momento de hacer operaciones. Se trata de utilizar un tipo en común para realizar la operación. Según la especificación del lenguaje para Java en su última versión (Java 13) Cap. 5, sección 6, punto 2 las conversiones se toman en el siguiente orden:

1. Si alguno de los operandos es double, el otro se convierte a double.
2. Si alguno de los operandos es float, el otro se convierte a float.
3. Si alguno de los operandos es long, el otro se convierte a long.
4. De lo contrario, ambos operadores son considerados int.

Para más información, se puede consultar la especificación en el siguiente enlace:

<https://docs.oracle.com/javase/specs/jls/se13/html/index.html> o descargarla desde aquí <https://docs.oracle.com/javase/specs/jls/se13/jls13.pdf>

Ejemplo:

```
byte unByte = 7;
byte otroByte = 9;
//el resultado deberá ser casteado, porque según las reglas
//ambos se consideran int
byte resultado = (byte) (unByte + otroByte);
```

Conversión de tipos de referencia

Si bien no vimos herencia hasta ahora, el concepto en su más básica expresión es sencillo. Una sub-clase hereda de otra super-clase sus atributos y operaciones. Por ejemplo, una Clase Ejecutivo heredaría todos los atributos y operaciones de una clase Empleado, que a su vez heredaría todo de una clase Persona. Entonces, un Ejecutivo se lo puede considerar un Empleado (de hecho, en la realidad lo es) y Empleado se lo puede considerar una persona (de hecho, en la realidad lo es). Por carácter transitivo, entonces, un Ejecutivo es una Persona.

Upcasting

La conversión de tipos “hacia arriba” se hace de manera implícita y es evidente si seguimos lo dicho más arriba:

```
Empleado e = new Empleado(); //'e' aquí guarda un objeto
Empleado

Empleado e2 = new Empleado(); //'e2' aquí guarda un objeto
Empleado

Persona p = new Persona(); //'p' aquí guarda una objeto
Persona

//entonces, según lo dicho es razonable poder hacer:
Persona x = e; //porque todo Empleado es Persona.

//podemos hacerlo explícitamente también:
Persona y = (Persona)e2;
```

Es importante notar que por más que ‘x’ e ‘y’ sean empleados, el tipo de la variable es Persona, por lo cual solo tendremos acceso a las características y operaciones de Persona.

Downcasting

Se trata de la conversión de tipos “hacia abajo”. De la superclase, hacia la subclase.

```
Persona p = new Persona(); //(*)
```

Esto no tiene nada de nuevo, pero si escribo esto:

```
Persona p2 = new Ejecutivo();
```

Esto está bien, ya que por la estructura jerárquica que describimos al principio el compilador es capaz de interpretar esto como correcto. Pero el camino inverso no necesariamente es cierto. Por ejemplo si tenemos:

```
Ejecutivo ej = new Persona();
```

Todo Ejecutivo es una Persona, pero no toda Persona es necesariamente un Ejecutivo.

Ahora bien, supongamos que hacemos lo siguiente:

```
Persona p3 = new Ejecutivo(); // (**)
```

Que no tiene nada de malo, ya que como vimos antes el compilador entiende esto como correcto. Si vemos acá hay una conversión de tipos implícita, es decir, hay un cast implícito.

Por otro lado, si hago:

```
Ejecutivo eje = p;
```

El compilador sabe que no necesariamente toda persona es un ejecutivo (alguna puede serlo), por lo que arrojará un error de compilación. Pero podemos decirle al compilador, “prometerle”, que efectivamente p almacenamos una instancia de Ejecutivo mediante un downcast explícito:

```
Ejecutivo eje2 = (Ejecutivo)p3;
```

InstanceOf

¿Qué pasa si con esto le estoy “mintiendo” al compilador?

```
//aca le prometo al compilador, y le estoy cumpliendo
Ejecutivo eje2 = (Ejecutivo)p3; //(ver **)

//aca estoy “mintiendo” porque en p (ver *) guardé una
persona
Ejecutivo eje = (Ejecutivo)p;
```

En este caso, no obtendremos un error en momento de compilación, sino en momento de ejecución del programa obtendremos un error de casteo, una `ClassCastException`. Es importante recordar que el compilador analiza la sintaxis del código, no lo ejecuta. Lo que ocurre durante la ejecución es distinto. Es decir, es válido hacer un casteo en este caso, pero ya dijimos que no necesariamente todas las personas son Ejecutivos (p3 en el caso de arriba es una persona que es Ejecutivo, pero p es una Persona que no lo es, en una simple persona que caminaba por la vereda de la empresa...).

¿Y si fue un error involuntario?

Antes de castear podemos asegurarnos de que puede hacerse correctamente. Para eso existe el operador `instanceof`. El operador `instanceof` no es un método, es un operador (como el + o el *), sirve para averiguar si una variable efectivamente esta almacenando una instancia de una clase u otra. El uso de `instanceof` es el siguiente:

```
miVariable instanceof MiClase;
```

Si `miVariable` está almacenando un objeto de tipo `MiClase`, entonces la operación devolverá un `true`. De lo contrario, un `false`.

Ejemplo:

```
if(p instanceof Ejecutivo) {  
    //siempre puedo castear, pero aca evito un error durante la  
    ejecucion  
    Ejecutivo eje = (Ejecutivo)p;  
}
```